LLMOps: DevOps for Data Scientists

Ali Gökçek
Advisor: Bahri Atay Özgövde
CMPE 492 — Department of Computer Engineering, Boğaziçi University

GitHub Repository

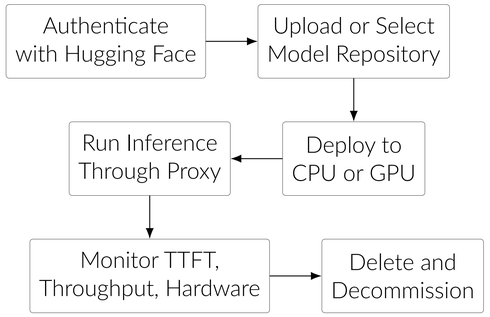
Motivation

Data scientists can fine-tune, upload, or discover language models, but production serving still demands cloud credentials, infrastructure provisioning, hardware selection, endpoint management, monitoring, and teardown. These operational tasks slow experimentation and make LLM deployment depend on DevOps expertise.

Goal: build a browser-based LLMOps platform that turns model ownership into a guided deploy, test, monitor, and decommission workflow.

Methodology: engineered with **GitHub Spec Kit**, executable specifications (`spec.md`, `plan.md`, `tasks.md`) are the source of truth, not ad-hoc patches.

Lifecycle Workflow



The platform keeps the workflow continuous: uploaded private models can be deployed directly, public or gated Hugging Face repositories can be served, and running deployments expose a copyable endpoint plus in-app inference.

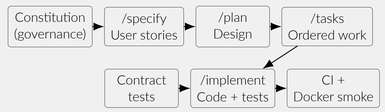
User-facing capabilities:

- **Authenticate & intake:** Hugging Face sign-in, local upload, and model selection.
- **Deploy & infer:** explicit CPU/GPU choice, live endpoint, and proxy inference.
- **Monitor & teardown:** fleet health, TTFT/throughput metrics, Grafana links, and deletion.

Spec-Driven Development (GitHub Spec Kit)

The project **constitution** (`.specify/memory/constitution.md`, v1.0.0) gates every feature. Six principles:

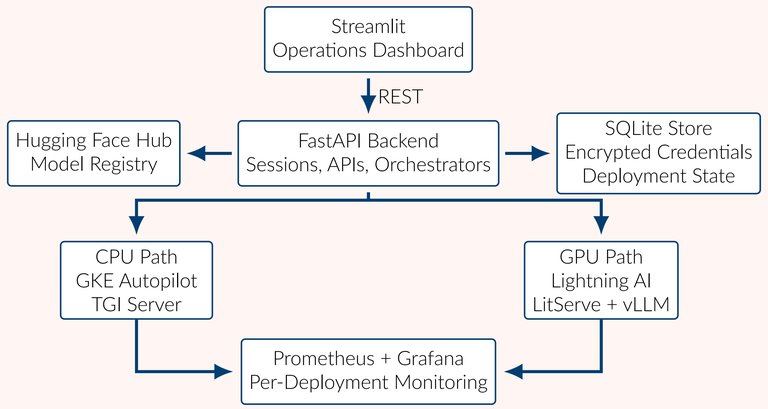
- **I. Clean & Readable Code:** self-explanatory names; minimal comments.
- **II. Security First:** no credential or token leakage; safe for open source.
- **III. Direct Framework Usage:** use FastAPI/Streamlit primitives, not wrappers.
- **IV. TDD Mandatory:** red-green-refactor; tests before implementation.
- **V. Realistic Testing:** contract tests, fake cloud providers in CI, real workflows.
- **VI. Simplicity:** minimal diffs; fix root causes, not symptoms.



When behavior drifts, fixes target the **specification** first.

Eight incremental specs (004–011): upload → session persistence → GKE CPU deploy → GPU path → HF cloud deploy → Prometheus/Grafana monitoring → ops dashboard refactor.

System Architecture

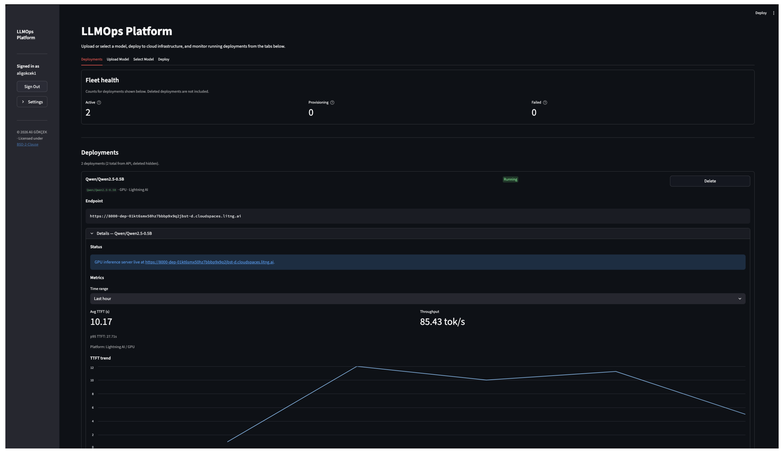


The backend accepts deployment requests immediately, then drives an asynchronous state machine from `queued` to `running`, `failed`, `lost`, or `deleted`. CPU deployments create dedicated GCP/GKE resources; GPU deployments are submitted to Lightning AI.

Security and Reliability Choices

- GCP service-account keys and Lightning AI API keys are encrypted at rest with Fernet.
- Hugging Face tokens remain session-scoped and are injected into deployments only as transient runtime environment variables.
- Grafana dashboards are opened through time-limited signed links scoped to the owning deployment.
- Uploads and deploys use idempotency and session continuity so long-running operations are not duplicated by re-login.

Operations Dashboard



Fleet overview: deployment status, hardware type, inference, and per-deployment metrics.

Engineering Contributions

- **Dual serving path:** one API routes CPU deployments to GKE/TGI and GPU deployments to Lightning AI/vLLM.
- **Unified inference proxy:** CPU and GPU endpoints return a common chat-style response contract.
- **Per-deployment observability:** TTFT, token throughput, and hardware metrics feed Prometheus and Grafana.

Monitoring (Grafana)

Per-deployment TTFT, throughput, and hardware utilization panels.

Validation

Area	Validation Evidence
Backend	FastAPI contract and integration tests
Frontend	Streamlit workflow and unit tests
Cloud safety	Fake GCP and Lightning providers in tests
CI	Pytest, Ruff, Docker build, smoke checks
Local demo	Docker Compose starts UI, API, Prometheus, Grafana

Outcome

Key result: one dashboard takes a Hugging Face model through CPU or GPU deploy, proxy inference, Prometheus/Grafana monitoring, and teardown—without a separate DevOps toolchain.

Data scientists can stage models on Hugging Face, choose infrastructure, receive a live inference endpoint, inspect service health, and tear down resources from a single operations UI.

Acknowledgments: Project guidance by Bahri Atay Özgövde and the Department of Computer Engineering, Boğaziçi University.